

Machine Learning from Zero

Everything in the pavement roughness project, explained for someone who has never met machine learning

A companion to the Code Guide and the Project Report

David Badu

September 2026

Contents

Before you start
Chapter 1: What is machine learning?
Chapter 2: The data, our raw material
Chapter 3: Looking at the data before learning anything
Chapter 4: The simplest model, a straight line
Chapter 5: Measuring how wrong a model is
Chapter 6: How the computer learns, gradient descent
Chapter 7: Testing honestly
Chapter 8: How good is good? Scores and baselines
Chapter 9: Too simple or too complex
Chapter 10: Better information beats a better model
Chapter 11: Trees and forests
Chapter 12: Yes or no questions, classification
Chapter 13: What we found, and what it teaches
Glossary

Before you start

This document assumes you know nothing about machine learning. It does not assume you are good at programming or statistics either. Every idea is introduced with an everyday example first, then connected to our project on road roughness, and only then described in the language used by machine-learning people.

You do not need to read the Code Guide first. This document explains the **ideas**; the Code Guide shows the **code**. Read this one to understand *why*, and the other to understand *how*.

Each chapter ends with a green box called **In one sentence**. If you only remember those boxes, you will have understood the project.

Chapter 1: What is machine learning?

1.1 Two ways to get a computer to do something

Normally, a person writes exact rules for the computer: *if the traffic light is red, stop*. This works when we know the rules. But for many problems we do not know the exact rules, or there are too many of them to write down.

Think about how an experienced highway engineer estimates how rough a road will be next year. Nobody gave them a formula. They have seen hundreds of roads, and from those examples they have developed a feel for which roads deteriorate fast. **Machine learning is teaching a computer to do the same thing: to learn a pattern from many examples instead of being given the rule.**

Traditional programming	Machine learning
A person writes the rules	The computer finds the rules from examples
Input: data + rules	Input: data + correct answers
Output: answers	Output: a rule (called a model) that can give answers for new data

1.2 Supervised learning

In **supervised learning**, the computer learns from examples where the correct answer is already known, as a student learns from worked examples with the answers at the back of the book. In our project, each example is one measurement of a road, and the known answer is how rough it was.

Everyday word	Machine-learning word	In our project
An example	Example, observation, row	One road section measured in one year
Things we know about the example	Features, inputs, x	Age, traffic, temperature, frost, pavement type
The answer we want to predict	Target, label, output, y	The road's roughness (IRI)
The rule learned from examples	Model	A formula that turns features into a predicted IRI
Learning from examples	Training	Adjusting the formula to fit past measurements

1.3 Two kinds of question

- **Regression:** the answer is a number. *How rough will this road be?* (for example 1.4 m/km).
- **Classification:** the answer is a category. *Will this road be in poor condition, yes or no?*

Our project does both: regression first (predicting IRI), then classification (predicting poor condition). These are exactly the two halves of the course *Supervised Machine Learning: Regression and Classification*.

In one sentence

Machine learning finds a rule from examples with known answers, so the rule can give answers for new examples; regression predicts numbers and classification predicts categories.

Chapter 2: The data, our raw material

2.1 What is IRI?

The **International Roughness Index (IRI)** measures how bumpy a road is. Imagine driving a standard car one kilometre and adding up how far its suspension moves up and down. That total, in metres per kilometre (m/km), is the IRI. A brand-new smooth road might be under 1 m/km; a badly worn road can be above 4 m/km. Lower is better.

US highway agencies use three condition bands: **Good** below 1.50 m/km, **Fair** from 1.50 to 2.68 m/km, and **Poor** above 2.68 m/km.

2.2 Where the data came from

The **Long-Term Pavement Performance (LTPP)** programme in the United States has monitored thousands of test road sections since 1987. Engineers go back to the same sections again and again to measure roughness, and the programme also records traffic and weather. This gave us:

- **15,935 measurements** after cleaning,
- on **1,550 road sections**,
- in **62 states and provinces**,
- from **1989 to 2025**.

2.3 The table

The data lives in a table, just like a spreadsheet. Each row is one road section in one year; each column is one piece of information about it.

Table 2.1. Three rows of the data (simplified)

Section	Year	Age (yrs)	Trucks/day	Temp (°C)	Frost index	IRI (m/km)
01_0502	1994	3	241	18.2	1	0.854
01_0502	1996	5	467	18.3	0	0.819
01_0502	1999	8	510	18.9	0	0.814

The last column is the target. All the others (and more) are candidate features.

2.4 The same road appears many times

Section 01_0502 appears 17 times, once for each survey year. On average each section appears about 10 times. Data like this, where the same things are measured over and over, is called **panel data**.

This matters a great deal, and we will come back to it. Two measurements of the same road a year apart are very alike. They are not independent examples, the way two different houses would be.

2.5 Cleaning

Real data is messy. Before learning anything we removed 35 duplicated rows, 10 rows with no traffic data, and 5 rows where the share of heavy trucks was above 100%, which is impossible. We also removed some columns that were really the answer in disguise: the roughness of the left and right wheel paths, whose average is the IRI. Leaving those in would be like giving a student the answers during an exam. This mistake is called **target leakage**.

In one sentence

Our examples are 15,935 yearly roughness measurements on 1,550 US test roads, each with traffic, climate and history information, and the same road appears many times.

Chapter 3: Looking at the data before learning anything

A good engineer inspects a site before designing. A good data analyst looks at the data before building a model. This stage is called **exploratory data analysis**.

3.1 A histogram: which values are common?

A **histogram** sorts all values into ranges and draws a bar for each range. The taller the bar, the more measurements fell in that range.

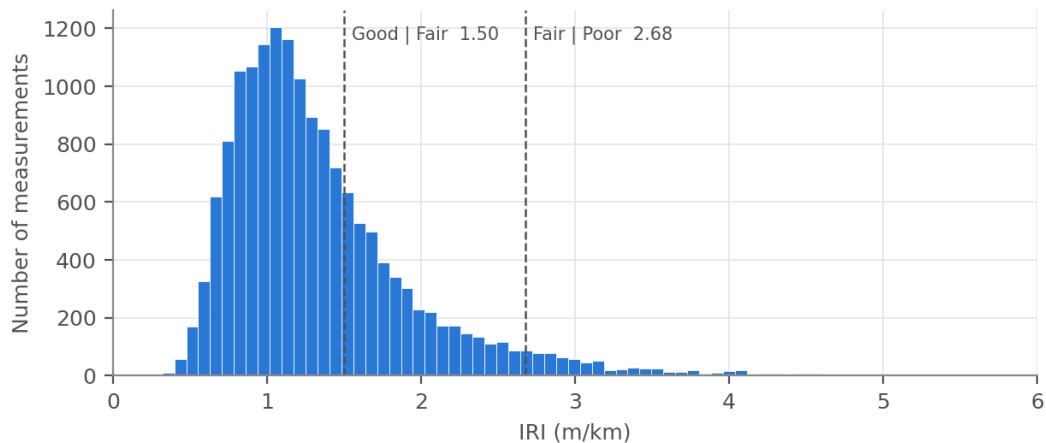


Figure 3.1. Histogram of IRI, with the Good, Fair and Poor thresholds.

Most roads are between about 0.7 and 1.6 m/km, but a thin tail stretches to the right: a few very rough roads. When one side has a long tail like this, the data is **skewed**.

3.2 Mean and median

The **mean** is the ordinary average. The **median** is the middle value when everything is sorted. For IRI, the mean is 1.35 m/km but the median is 1.20 m/km. The few very rough roads pull the mean up, but they do not move the middle value. When data is skewed, the median describes a *typical* road better.

3.3 A useful trick: the logarithm

Taking the **logarithm** of a number shrinks big numbers much more than small ones: $\log(1) = 0$, $\log(2) = 0.69$, $\log(5) = 1.61$. Applied to IRI, it pulls in the long tail and makes the histogram almost symmetric (skewness fell from 1.71 to 0.35). We tried predicting $\log(\text{IRI})$ instead of IRI; in the end it made predictions worse, so we kept plain IRI. The lesson: try ideas, but let the results decide.

3.4 Scatter plots and trends

A **scatter plot** draws one dot per row, with a feature across the bottom and IRI up the side. If the dots drift upward, the feature and IRI rise together.

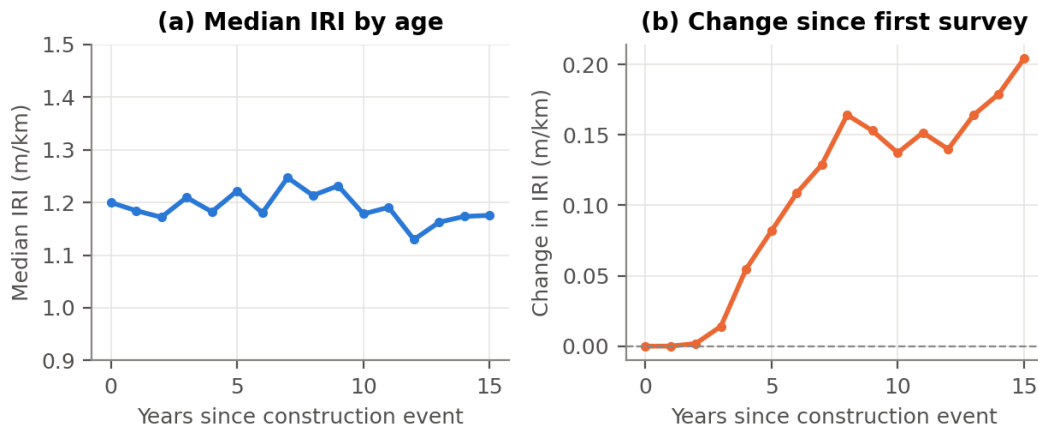


Figure 3.2. (a) Median IRI at each age is almost flat. (b) But each road's own IRI rises over time.

Figure 3.2 shows something surprising and important. When all roads are mixed together (a), older roads do not look rougher. But when we follow each road over its own life (b), roughness clearly grows, by about 0.2 m/km over 15 years. Why the difference?

- **Rough roads get repaired.** When a road gets too rough it is rehabilitated and its age resets to zero. So the roads that reach old age are mostly the ones that stayed smooth. This is called **survivorship bias**.
- **Every road starts differently.** Some are built smooth, some less so. Those differences between roads are bigger than the change within one road, and they hide it.

3.5 Correlation: one number for a relationship

The **correlation coefficient**, written r , summarises how strongly two things move together in a straight line. It runs from -1 (one goes up exactly as the other goes down) through 0 (no straight-line link) to $+1$ (they go up together exactly). Roughly, below 0.3 in size is weak, 0.3 to 0.7 moderate, and above 0.7 strong.

Every feature in our data had a **weak** correlation with IRI, none larger than about 0.23 in size. Traffic even had a slightly *negative* correlation: busier roads were a little smoother. That seems backwards, until you remember that busier roads are designed thicker and maintained more often. When a hidden factor (here, design and maintenance) affects both things, it is called **confounding**.

In one sentence

Looking first showed that IRI is skewed, that each road roughens over time even though old and young roads look similar overall, and that no single feature is strongly linked to roughness.

Chapter 4: The simplest model, a straight line

4.1 One feature

Suppose we only use age to predict IRI. The simplest rule is a straight line:

$$f_{w,b}(x) = wx + b$$

- x is the input (age),
- $f(x)$ is the predicted IRI,
- w (the *weight*) is the slope: how much IRI rises per year of age,
- b (the *bias* or intercept) is the starting value: predicted IRI at age 0.

For example, with $w = 0.1$ and $b = 0.8$, a 5-year-old road is predicted to have $\text{IRI} = 0.1 \times 5 + 0.8 = 1.3$ m/km.

Learning means finding the values of w and b that make the line fit the examples as well as possible. The numbers w and b are called the **parameters** of the model.

4.2 Many features

With several features, each gets its own weight, and the prediction adds them all up:

$$f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

For our project with 10 features, the model is: predicted IRI = $w_1 \times \text{age} + w_2 \times \text{rehabilitated} + w_3 \times \text{rigid} + \dots + w_{10} \times \text{freeze-thaw} + b$. Learning means finding 10 weights and one b . This is **linear regression**: *linear* because it is a weighted sum (a straight line, or flat surface), *regression* because it predicts a number.

4.3 How to read a weight

A weight tells you how much the prediction changes when that feature goes up by one unit, while all other features stay the same. A positive weight means the feature pushes roughness up; a negative weight pushes it down. In our model, **rigid pavement had the largest positive weight**, matching what we saw in the data (concrete sections were rougher).

In one sentence

Linear regression predicts a number as a weighted sum of the features plus a starting value, and learning means finding the best weights.

Chapter 5: Measuring how wrong a model is

5.1 Errors

To find the *best* line, we first need a way to score how bad a line is. For each road, the **error** is the prediction minus the true value. A good line has small errors.

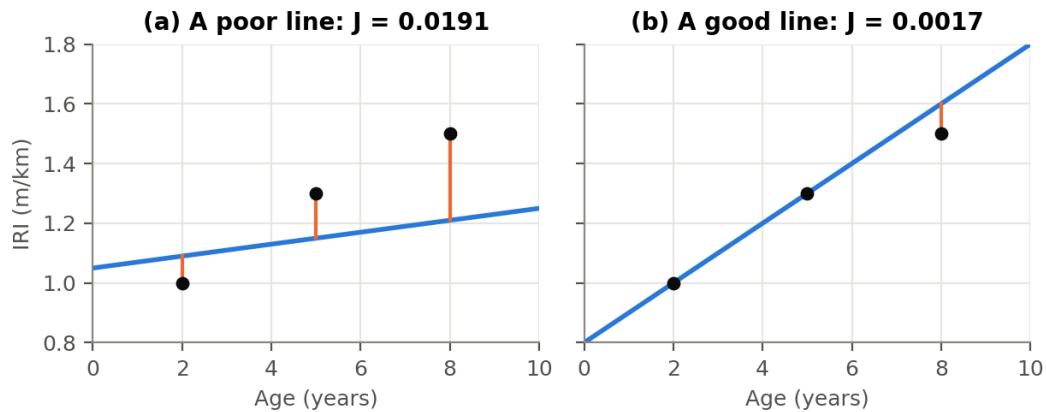


Figure 5.1. Three roads (black dots). The orange lines are the errors. The line in (b) fits much better.

5.2 The cost function

We combine all the errors into a single score called the **cost**, written J . The usual choice squares each error (so negatives do not cancel positives, and big errors count extra), adds them up, and divides by $2m$, where m is the number of examples:

$$J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\mathbf{w}, b}(\mathbf{x}^{(i)}) - y^{(i)})^2$$

The symbols look heavy, but the idea is simple: **J is basically the average squared error**. Small J means a good fit.

5.3 A worked example with three roads

Table 5.1. Three example roads

Road	Age x	True IRI y
1	2	1.0
2	5	1.3
3	8	1.5

Try $\mathbf{w} = \mathbf{0}$ and $b = 0$. Every prediction is 0. The errors are -1.0 , -1.3 and -1.5 . Squared: 1.00, 1.69, 2.25. Sum: 4.94. Divide by $2 \times 3 = 6$: **$J = 0.823$** . That is bad.

Try $\mathbf{w} = 0.1$ and $b = 0.8$. Predictions: 1.0, 1.3 and 1.6. Errors: 0, 0, 0.1. Squared sum: 0.01. Divide by 6: **$J = 0.0017$** . Much better.

So choosing good values of $\mathbf{w}$ and b makes J small. But with 10 features we cannot just guess. We need a method.

In one sentence

The cost J is the average squared error of the predictions, and the best model is the one with the smallest J .

Chapter 6: How the computer learns, gradient descent

6.1 Walking downhill in fog

Picture J as a landscape: every possible choice of weights is a place, and the height there is the cost. We want the lowest valley. Imagine standing on a hillside in thick fog. You cannot see the valley, but you can feel which way the ground slopes under your feet. So you take a step downhill, feel again, take another step, and keep going until the ground is flat. That is **gradient descent**.

- The **gradient** is the slope under your feet: which direction makes the cost go up, and how steeply.
- Each step moves the weights a little in the **opposite** direction (downhill).
- The **learning rate**, written α (alpha), is the size of each step.

$$w_j := w_j - \alpha \frac{\partial J}{\partial w_j} \quad b := b - \alpha \frac{\partial J}{\partial b}$$

The symbol $:=$ means "replace with". Each step, every weight is nudged downhill. After enough steps, the weights settle at the bottom of the valley.

6.2 One step of the worked example

Start at $w = 0$, $b = 0$, with $J = 0.823$. The slope works out to -6.83 for w and -1.27 for b (negative means "going up in w makes the cost go down"). With a learning rate of 0.01 , the new values are $w = 0 - 0.01 \times (-6.83) = \mathbf{0.068}$ and $b = 0 - 0.01 \times (-1.27) = \mathbf{0.013}$. The cost drops from 0.823 to $\mathbf{0.417}$ after a single step. Repeat hundreds of times and it keeps falling.

6.3 Choosing the step size

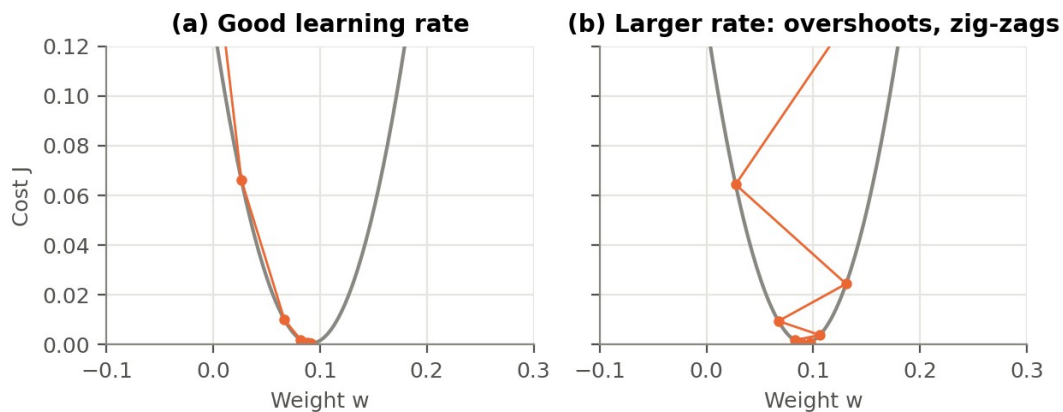


Figure 6.1. Each orange dot is one step. (a) Sensible steps walk to the bottom. (b) Bigger steps overshoot and zig-zag; even bigger ones fly off and the cost explodes.

Learning rate on our data	What happened
0.001	Too small: after 1,000 steps it still had not reached the bottom (cost 0.294)
0.01 to 0.5	Reached the bottom (cost 0.1701)
1.0	Too large: the steps overshoot further each time and the cost became infinite

6.4 Scaling the features first

Our features have very different sizes: the freezing index reaches 3,647 while the share of heavy trucks stays below 1. That makes the valley long and narrow, and gradient descent zig-zags across it slowly. The cure is **feature scaling**: rescale every feature so its average is 0 and its typical spread is 1:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

where μ (mu) is the feature's average and σ (sigma) its standard deviation (typical distance from the average). After scaling, all features are on similar footing, the valley becomes round, and gradient descent walks straight down. A bonus: scaled weights can be compared with each other directly.

6.5 Did it work?

We wrote gradient descent ourselves, ran it for 1,000 steps, and then asked the scikit-learn library to solve the same problem its own way. The weights matched to four decimal places. That is how we know our own code is correct.

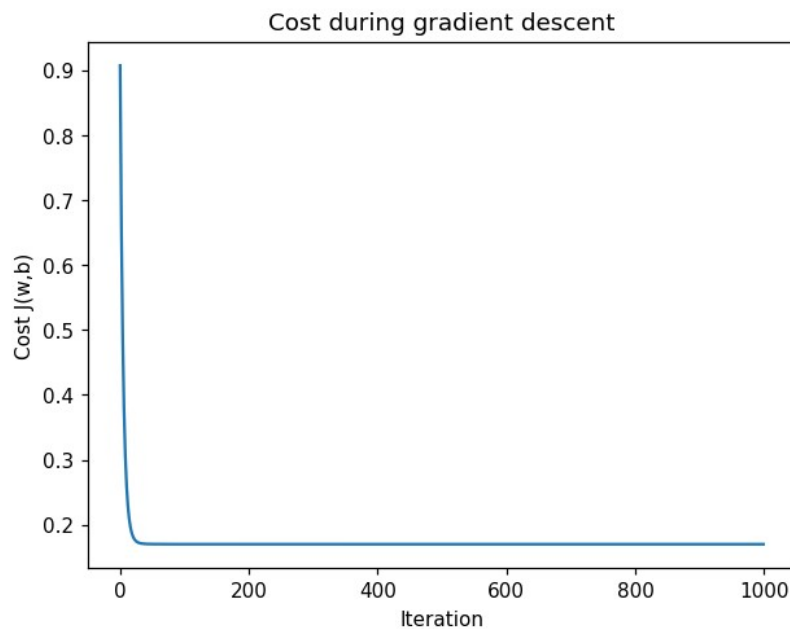


Figure 6.2. On our data, the cost dropped quickly and then flattened out: the model had converged.

In one sentence

Gradient descent repeatedly nudges the weights downhill on the cost landscape, with a step size (learning rate) that must be neither too small nor too large, and it works best when features are scaled.

Chapter 7: Testing honestly

7.1 The exam analogy

A model that has learned from certain examples will naturally do well on those same examples. That proves nothing, just as a student who has memorised last year's exam paper is not proven to understand the subject. What we really want to know is: **how well will it do on roads it has never seen?**

So before learning, we lock away part of the data as a **test set** (about 20%). The model learns only from the **training set** (about 80%). At the very end, we use the test set once to measure performance.

7.2 Split by road, not by row

Because each road appears about 10 times, a careless split could put a road's 1994 survey in training and its 1996 survey in testing. The model would effectively have seen the answer. So we split by **road**: all surveys of a road go to the same side.



Figure 7.1. Splitting by road: every survey of roads 9 and 10 is kept out of training.

7.3 Hidden sisters: sites and locations

Later we discovered something subtler. LTPP often builds several test sections side by side at one site, to compare designs. Those sister sections share the same traffic and the same weather. If one sister is in training and another in testing, a clever model can recognise the shared traffic and weather pattern and "remember" the site. So in the end we grouped by **location**: sections sharing the same weather record always stay together.

This mattered enormously. One model (a random forest, Chapter 11) appeared to explain about 35% of the variation when split by road, about 9% when split by site, and about 0% when split by location. Almost all of its apparent skill came from recognising places, not from understanding roads. Any accidental sharing of information between training and test data is called **data leakage**.

7.4 Cross-validation

One split can be lucky or unlucky. **Cross-validation** divides the training data into 5 parts (folds), trains on 4 and tests on the fifth, five times over, so each part takes one turn as the test. The average of the five scores is more reliable. We used it to compare models and choose settings, without ever touching the final test set.

In one sentence

A model is only judged on data it never saw, and because roads, sites and locations share information, the split must keep related data together or the score will be falsely good.

Chapter 8: How good is good? Scores and baselines

8.1 Three ways to score a regression model

Score	Plain meaning	Units
MAE (mean absolute error)	The average size of a mistake	m/km
RMSE (root mean squared error)	Like MAE, but big mistakes count more	m/km
R^2 (R-squared)	The share of the ups and downs in IRI that the model explains. 1 = perfect, 0 = no better than always guessing the average, negative = worse than that	none

8.2 Always compare with a baseline

A score on its own means little. We always compare with the simplest possible prediction, called a **baseline**. For regression, the simplest is to predict the *average* IRI for every road ($R^2 = 0$).

Our first linear model, using traffic, climate and section information, had a test RMSE of 0.595 m/km against 0.619 for the average. **An improvement of only about 4%.** In R^2 terms it explained under 10% of the variation, and under the strictest location-based testing about 4%.

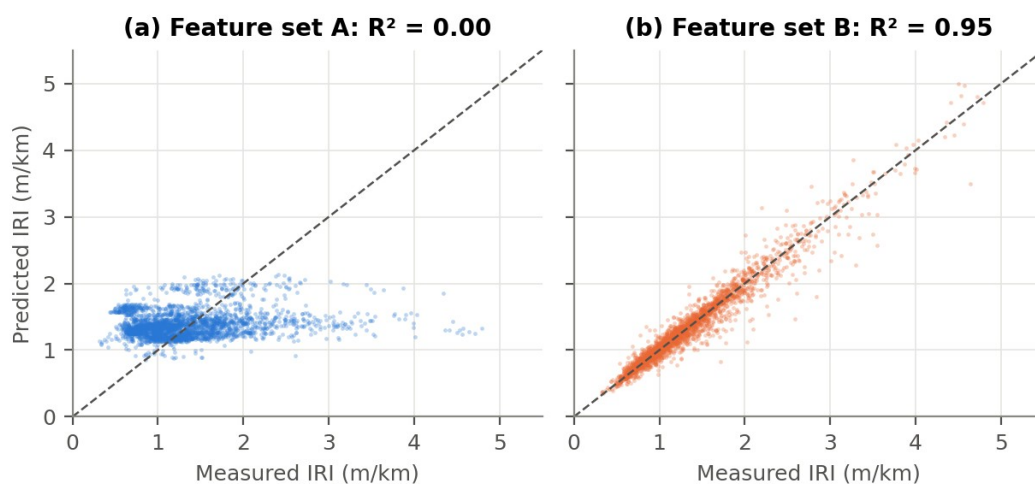


Figure 8.1. Predicted against measured IRI for roads never seen in training. (a) Traffic and climate only: the model predicts roughly the same value for every road. (b) Adding previous IRI (Chapter 10).

8.3 Residuals

A **residual** is measured minus predicted. Plotting residuals shows *where* the model goes wrong. Ours had a long tail of big positive residuals: very rough roads that the model predicted as average. It could not tell a bad road from a good one.

In one sentence

Scores such as MAE, RMSE and R^2 only mean something next to a simple baseline, and our first model barely beat guessing the average.

Chapter 9: Too simple or too complex

9.1 Underfitting and overfitting

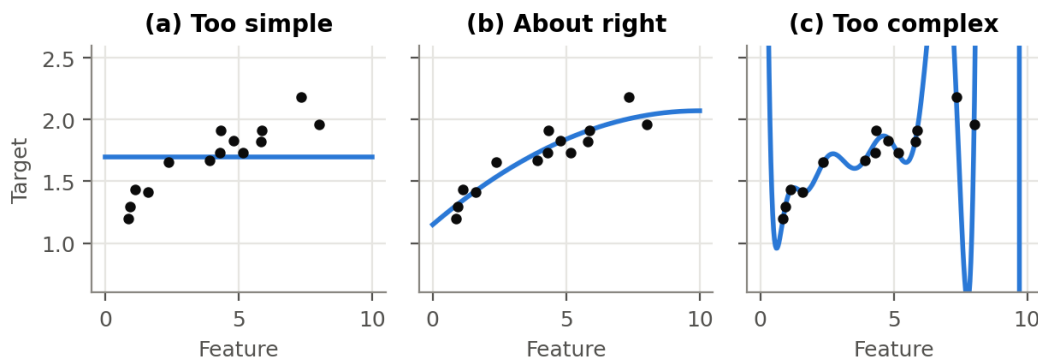


Figure 9.1. The same points fitted three ways.

- **Underfitting** (a): the model is too simple to capture the pattern. It does badly on training data and on new data. Also called **high bias**.
- **Good fit** (b): captures the real pattern, ignores the noise.
- **Overfitting** (c): the model is so flexible it follows every random wiggle in the training data. It looks perfect on training data but fails on new data. Also called **high variance**.

A student who only memorises one formula underfits; a student who memorises every past exam answer word for word overfits. We want the student who understands.

9.2 Making the model more flexible: polynomial features

A straight line cannot bend. To let linear regression draw curves, we add new features made from the old ones: age squared, age times frost, and so on. These are **polynomial features**. With 10 features, degree 2 gives 65 columns, degree 3 gives 285, and degree 4 gives 1,000.

Table 9.1. More flexibility: training score rises, test score collapses

Degree	Training R^2	Test R^2	Verdict
1 (straight)	0.100	0.068	Underfitting
2	0.155	0.073	Slightly better
3	0.244	0.020	Starting to overfit
4	0.364	-1.87	Badly overfitting

At degree 4 the model was **far worse than guessing the average** on new roads. This is the **bias and variance trade-off**: more flexibility reduces bias but raises variance, and the best point is in between.

9.3 Regularisation: a penalty for complexity

Regularisation keeps a flexible model but punishes it for using large weights, because large weights are what let a model twist itself around noise. A setting called λ (lambda) controls the strength of the penalty: $\lambda = 0$ means no penalty, a huge λ squashes all weights towards zero.

$$J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\mathbf{w}, b}(\mathbf{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

With regularisation, the overfitting degree-3 model came back under control. But even the best version was no better than the plain straight line on new roads. **The problem was not the shape of the model; the features simply did not contain the information.** No amount of curve-fitting can extract information that is not there.

In one sentence

Too simple a model underfits and too complex a model overfits; regularisation tames complexity, but it cannot create information the features do not hold.

Chapter 10: Better information beats a better model

10.1 The best clue: last time's roughness

Ask an experienced engineer to guess a road's roughness next year, and the first thing they will want to know is: *how rough is it now?* Roads change gradually. So we added two features: **previous IRI** (the IRI at the last survey of the same road) and **gap** (the years since that survey). A feature built from an earlier value of the same series is called a **lagged feature**.

The effect was dramatic: R^2 on new locations jumped from about 0.04 to about **0.93**, and the typical error fell from about 0.42 to about 0.09 m/km.

10.2 The persistence check

A result that good deserves suspicion. What would the laziest possible forecast score? The laziest forecast is **persistence**: *next year will be the same as last time*. No learning at all.

Table 10.1. Five-fold cross-validation, grouped by location

Forecast	R^2 on new locations	RMSE (m/km)
Average IRI for every road	-0.02	0.592
Traffic and climate model	0.04	0.575
Persistence (same as last time)	0.91	0.178
Model with previous IRI	0.93	0.160

Persistence alone scores 0.91. So nearly all of the impressive 0.93 comes from simply knowing last time's value. What traffic, climate and the time gap add is the step from 0.91 to 0.93, cutting the error by about 10%. And when we asked the model to predict the **change** in roughness between surveys, it explained only about 5% of it.

10.3 What still helps

Once the model knows a road's previous condition, the features that help predict *how much more* it roughens are: the **time since the last survey** (more time, more deterioration), the **freezing index** (frost damage) and the **temperature**. This fits pavement engineering knowledge.

In one sentence

Knowing a road's previous roughness predicts its current roughness very well, but most of that is just persistence; predicting how fast roads deteriorate is still hard with this data.

Chapter 11: Trees and forests

11.1 A decision tree is a game of twenty questions

A **decision tree** predicts by asking a series of yes/no questions, like a flowchart: *Is the previous IRI below 1.4? If yes, is the gap one year or less?* At the end of each branch (a **leaf**) it predicts the average IRI of the training roads that ended up there. The tree picks each question to reduce the error as much as possible.

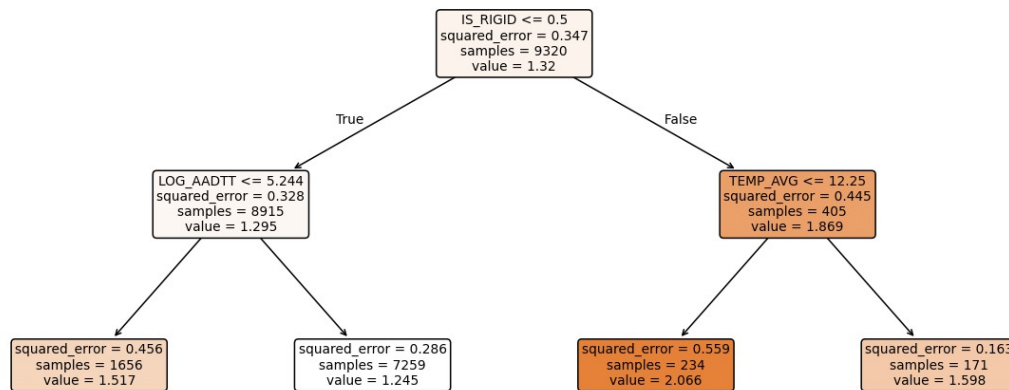


Figure 11.1. A small decision tree learned from our data (two levels of questions).

Trees need no scaling and can find curves and combinations by themselves (for example "frost matters only for rigid pavements"). Their complexity dial is **depth**: how many questions in a row. A very deep tree memorises the training data and overfits, exactly like the degree-4 polynomial.

11.2 A random forest is the wisdom of the crowd

A single tree is unstable. A **random forest** grows hundreds of trees, each on a random sample of the data and considering random subsets of features, and averages their answers. Like averaging the estimates of 300 engineers, the individual mistakes tend to cancel out.

Forests are powerful, and that is exactly why they exposed the leakage problem in Chapter 7: they were clever enough to recognise sites from their traffic and weather. Under honest location-based testing, the forest did no better than the straight line.

11.3 Which features matter? Permutation importance

To ask a model which features it relies on, we shuffle one feature's values randomly (breaking its link with IRI) and measure how much the score drops. A big drop means the model depended on that feature. This is **permutation importance**.

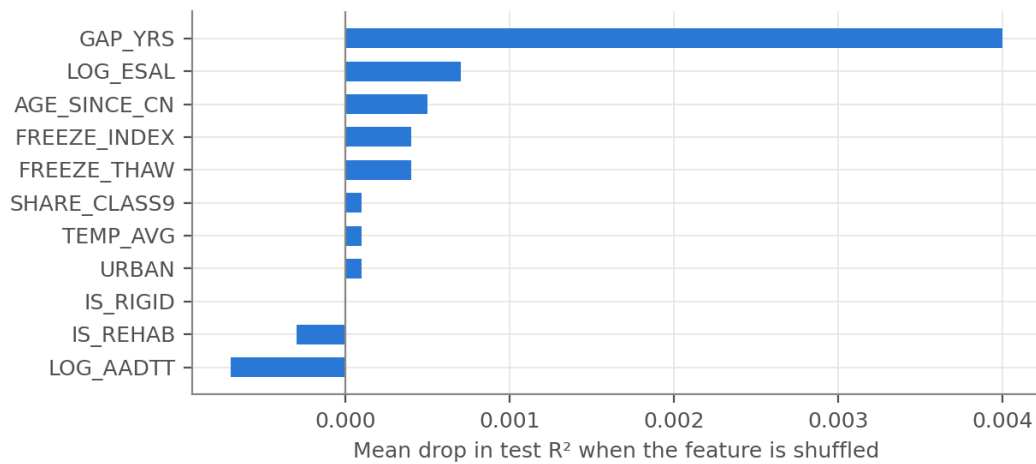


Figure 11.2. Permutation importance for the forest with previous IRI (previous IRI itself, by far the largest, is left out so the others are visible).

Previous IRI dwarfed everything. Among the rest, the gap between surveys was clearly the most useful. The linear model agreed. When two very different models agree, we can trust the finding more.

In one sentence

Trees ask questions and forests average many trees; they are flexible and powerful, which also makes them good at exploiting leakage, and they agreed that previous condition and time since the last survey matter most.

Chapter 12: Yes or no questions, classification

12.1 From a number to a probability

Now the question is: *will this section be in poor condition (IRI above 2.68 m/km)?* We want a probability between 0 and 1. **Logistic regression** takes the same weighted sum as linear regression and passes it through an S-shaped curve called the **sigmoid**, which squashes any number into the range 0 to 1.

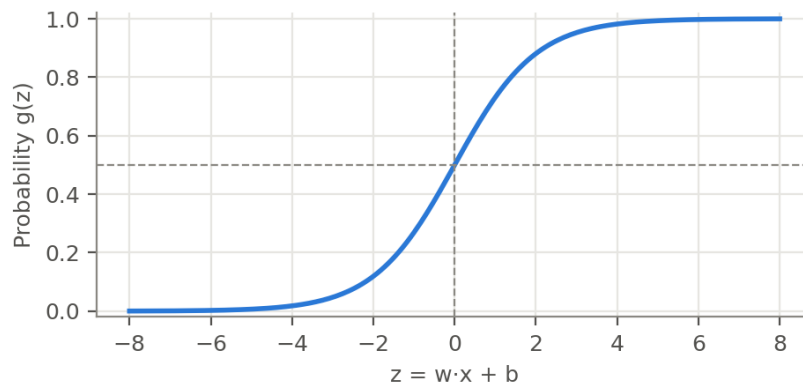


Figure 12.1. The sigmoid. Very negative sums give probabilities near 0, very positive sums near 1, and a sum of 0 gives exactly 0.5.

$$g(z) = \frac{1}{1 + e^{-z}} \quad z = \mathbf{w} \cdot \mathbf{x} + b$$

For example, a sum of -4 gives a probability of 0.02 (almost certainly not poor), 0 gives 0.5, and +2 gives 0.88. A **threshold**, 0.5 by default, turns the probability into a yes or no.

The cost is different too: **log loss** punishes confident wrong answers very heavily (saying 99% "not poor" for a road that is poor costs a lot). Otherwise training is the same gradient descent as before. We wrote it ourselves again, and our weights closely matched scikit-learn's.

12.2 The accuracy trap

Only about 4% of measurements are poor. A useless model that always says "not poor" would be right about 96% of the time, yet it would never find a single poor road, and finding poor roads is the whole point. When one class is rare, **accuracy is misleading**.

12.3 The confusion matrix, precision and recall

	Model says: not poor	Model says: poor
Really not poor	Correct (true negative)	False alarm (false positive)
Really poor	Missed poor road (false negative)	Correct (true positive)

- **Precision:** of the roads the model flagged as poor, how many really were? (Are we wasting inspection trips?)
- **Recall:** of the roads that really were poor, how many did the model catch? (Are we missing problems?)

Table 12.1. Classification on a held-out set of locations

Approach	Precision	Recall	Poor roads missed (of 107)	False alarms
Always "not poor"	0.00	0.00	107	0
Persistence (poor last time)	0.93	0.71	31	6
Logistic regression, threshold 0.5	0.88	0.83	18	12
Logistic regression, threshold 0.3	0.74	0.90	11	33
Logistic regression, threshold 0.1	0.52	0.96	4	97
Without previous IRI	0.06	0.63	40	1,132

12.4 The threshold is an engineering decision

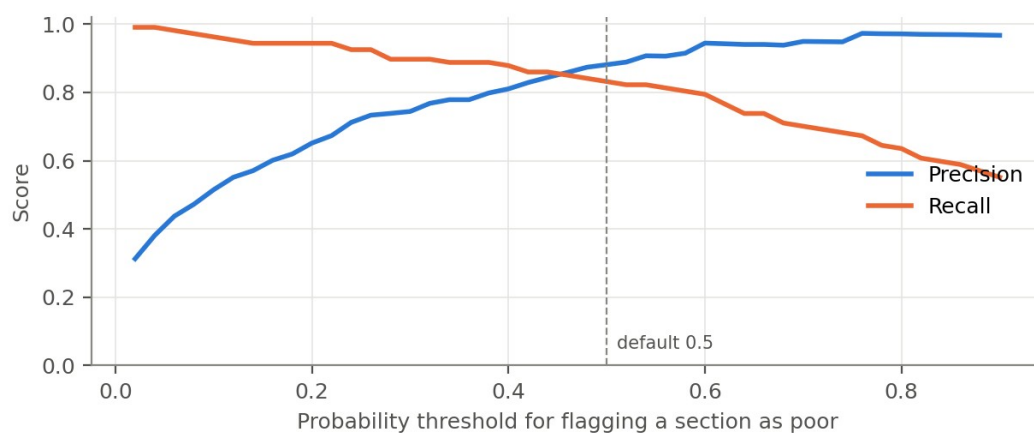


Figure 12.2. Lowering the threshold catches more poor roads (recall rises) at the cost of more false alarms (precision falls).

There is no free lunch. Missing a poor road means it keeps deteriorating and costs more to fix later; a false alarm means an unnecessary inspection. Choosing the threshold is a question of **budget and consequences**, not statistics. That is a decision for the highway agency.

And again, without previous IRI, traffic and climate alone could not pick out poor roads: only 6% of flagged roads were really poor.

In one sentence

Logistic regression turns a weighted sum into a probability; with rare classes we judge it by precision and recall, and the threshold trades missed problems against false alarms.

Chapter 13: What we found, and what it teaches

13.1 The findings

1. **Traffic, climate and section attributes alone predict very little of pavement roughness.** Under honest testing they explained about 4% of the variation.
2. **A road's previous roughness dominates.** Persistence alone explains about 91%; the other features add about 2 points and cut the error by about 10%.
3. **How we test changes the answer.** A random forest looked like it explained 36% of the variation, but almost all of that came from recognising locations it had already seen.
4. **The useful physical signals are consistent with engineering knowledge:** time since last survey, frost and temperature affect how much roads roughen, and rigid pavements are rougher on average.

13.2 Why traffic and climate were weak

- The data does not include the pavement **structure** (layer thicknesses, materials), which is what really controls how a road resists traffic.
- Busier roads are **built and maintained** to cope, which hides the damage traffic causes (confounding).
- Most of the difference in roughness is **between roads** (about 62% of the variation), driven by how each road was built.

13.3 Habits worth keeping

- Look at the data before modelling.
- Split by the right unit (road, site, location), never by random rows when data is grouped.
- Always compare with the simplest sensible baseline.
- Be suspicious of results that look too good, and test your suspicion.
- Choose settings with cross-validation, and use the test set once.
- Report weak results honestly; they are findings too.

In one sentence

The project showed, using every tool from the course and a few beyond it, that previous condition dominates pavement roughness prediction and that honest testing matters as much as the choice of model.

Glossary

Term	Meaning
Baseline	The simplest sensible prediction (for example the average), used as a yardstick
Bias (of a model)	Error from a model being too simple; causes underfitting
b (bias term, intercept)	The starting value added to the weighted sum
Classification	Predicting a category, such as poor or not poor
Confounding	A hidden factor that affects both a feature and the target, making their link misleading
Confusion matrix	Table of correct and incorrect yes/no predictions
Correlation (r)	A number from -1 to +1 for how strongly two things move together in a straight line
Cost function (J)	A single score of how wrong the model is; learning tries to make it small
Cross-validation	Repeating training and testing on different parts of the data and averaging
Data leakage	Information from the test data sneaking into training, making scores falsely good
Decision tree	A model that predicts by asking a series of yes/no questions
Feature (x)	An input used to make a prediction
Feature scaling	Rescaling features to similar ranges, usually mean 0 and standard deviation 1
Gradient	The slope of the cost: which way is uphill and how steep
Gradient descent	Learning by repeatedly stepping the weights downhill on the cost
IRI	International Roughness Index, in m/km; lower is smoother
Lagged feature	An earlier value of the same measurement, such as previous IRI
Learning rate (α)	Size of each gradient descent step
Linear regression	Predicting a number as a weighted sum of features plus b
Logistic regression	Predicting a probability by passing a weighted sum through the sigmoid
Log loss	The cost used for logistic regression
MAE, RMSE	Average size of errors; RMSE counts big errors more
Model	The rule learned from data
Overfitting	Fitting the training data so closely, noise included, that new data is predicted badly
Panel data	Data where the same units are measured repeatedly over time
Permutation importance	How much a model's score drops when one feature is shuffled
Persistence	Forecasting that the next value equals the last one
Polynomial features	New features made by squaring and multiplying existing ones
Precision	Of the items flagged, the share that are truly positive
R^2	Share of the variation in the target explained by the model
Random forest	Many decision trees averaged together
Recall	Of the truly positive items, the share that were caught
Regression	Predicting a number
Regularisation (λ)	A penalty on large weights to prevent overfitting
Residual	Measured value minus predicted value
Sigmoid	S-shaped function that turns any number into a probability between 0 and 1
Skewness	How lopsided a distribution is; positive means a long right tail

Term	Meaning
Survivorship bias	Distortion from only seeing the cases that "survived" some process
Target (y)	The value we want to predict
Test set	Data kept aside and used once to judge the final model
Threshold	The probability above which a classifier says yes
Training set	Data the model learns from
Underfitting	A model too simple to capture the real pattern
Variance (of a model)	Error from a model being too sensitive to its training data; causes overfitting
Weight (w)	The number multiplying a feature in the model